VaaniLedger

A voice- and chat-driven khata for small Indian shopkeepers

Technical report: architecture, decisions, and the reasoning behind them

189	1.000	122	20+
automated tests	macro F1	labelled cases	defects found

Capstone project · team of four · 29 August 2026

7,483 lines across a Python backend, an Android client and a web dashboard · 24 commits

READ THIS FIRST

Executive summary

VaaniLedger turns a WhatsApp order into a ledger entry without the shopkeeper typing anything. A floating bubble sits on top of WhatsApp; he taps it while reading a customer's message, and the app reads the chat off the screen, extracts the transaction, and asks for one tap to confirm.

The project is complete end to end on a device: capture, extraction, a confirm sheet with per-field confidence, an editable correction path that feeds a learning loop, an offline-first ledger, idempotent sync to a server, a web dashboard, assisted payment reminders, and a spoken daily summary. The extraction engine carries 189 automated tests and is measured against a hand-labelled dataset rather than asserted to work.

This report is organised around decisions rather than features. Nine of them shaped the product, and for each one it records what was at stake, what the alternatives were, what we chose and — most usefully — what that choice cost. A decision log that records only outcomes is a list, not reasoning.

THE SINGLE MOST CONSEQUENTIAL DECISION

The original design used a WhatsApp Business bot. We abandoned it in week one, because a verified WhatsApp Business Account requires business documentation and takes weeks to approve with no guaranteed date.

That would have placed a third party's approval process on the critical path of a project with a fixed, graded deadline. No amount of good engineering downstream recovers from a dependency like that. Everything distinctive about the product — the overlay, the on-screen capture, the assisted reminders — follows from removing it.

The other honest headline is about the accuracy figure. The engine scores 1.000 macro F1 on our 122-case dataset, and that number means the dataset is saturated, not that the engine is finished. We wrote the cases, then fixed the engine until it passed them. Section 11 explains exactly what the number does and does not support, because a reader who discovers that themselves discounts everything else in the report.

Contents

The problem and the market

1 Who this is for, and what it costs them today · 4
2 Why existing ledger apps fail commercially · 5
3 The insight the product rests on · 6

The decision log

4 Nine decisions and the reasoning behind each · 7

Technical architecture

5 System overview · 13
6 The extraction engine · 14
7 Hindi numbers, in both directions · 16
8 Reading the chat · 17
9 Bulk ingestion and deduplication · 19
10 The ledger, sync and khata aging · 20

Measurement and quality

11 How we measure, and what the numbers mean · 22
12 Every defect, and its lesson · 24

Commercial analysis

13 Unit economics and cost model · 28
14 Distribution, and why the Play Store is closed · 29

State, risk and roadmap

15 What is true today · 30
16 Risks and limitations we are not hiding · 31
17 Roadmap · 32
18 How to run it · 33

PART I · SECTION 1

Who this is for, and what it costs them today

A kirana is the neighbourhood grocery shop that most Indian households buy from daily. India has roughly 12 million of them. The defining commercial fact about one is that it runs on credit.

A regular customer does not pay per visit. He takes goods through the week or the month and settles later — this is udhaar, and for a typical kirana it is not a minor line item but a large share of turnover. The shop is therefore running an unsecured lending business alongside a retail one, with no software, no credit assessment, and no systematic collection process.

The record of all this is a paper notebook. That notebook is the shop's entire accounts receivable, and it has properties no business would accept if it were framed as a system design:

- Single point of failure. One notebook, no copy. Water, fire or simple loss destroys the record of every rupee owed.
- Unqueryable. Answering “how much does Rahul owe?” means leafing through months of entries by hand, so in practice it is answered from memory.
- No collection discipline. There is no list of who is overdue and by how long, so chasing depends on the shopkeeper happening to remember at the moment the customer is standing in front of him.
- Disputes are unwinnable. If a customer disagrees with a figure, there is a handwritten line and two memories.

WHAT THE PAIN ACTUALLY IS, COMMERCIALLY

The cost is not the bookkeeping effort. It is working capital trapped in uncollected udhaar, plus the amount silently written off because nobody remembered to ask.

That reframing matters for the product: the winning feature is not a prettier ledger, it is making collection systematic — which is why overdue flagging and assisted reminders are core rather than nice-to-have.

PART I · SECTION 2

Why existing ledger apps fail commercially

Digital khata apps exist, some with very large download numbers. Shopkeepers install them and then largely stop using them. Understanding why is the whole basis of our design.

Failure mode	Why it kills adoption
Data entry at the worst possible moment	The entry must be made mid-rush with a queue waiting. An app that asks for a form costs perhaps thirty seconds; a notebook costs three. The notebook wins every single time, and one skipped entry makes the whole ledger untrustworthy.
English-first, form-shaped interfaces	The user thinks and speaks in Hindi or Hinglish. A form with labelled fields imposes a mental model from accounting software on someone who has never used accounting software.
Double entry of data that already exists	The order arrived as a WhatsApp message. Asking him to read it and retype it is asking him to do the same work twice.
Trust asymmetry	The notebook is his. An app that silently records numbers he cannot verify is a worse instrument for money he is owed, not a better one.

Every one of these is a friction problem, not a feature gap. The implication is that adding features cannot win; the only thing that can win is removing the typing. That constraint set the entire product direction, and it is why the capture path — not the ledger — received the majority of the engineering effort.

PART I · SECTION 3

The insight the product rests on

The order is already on his phone.

Customers do not phone in orders any more; they send them on WhatsApp. A real one looks like this:

```
"Bhaiya 2 kilo chawal aur 1 packet chai bhej do,
 dhai sau ka udhaar likh lena"

  who      -> the WhatsApp contact who sent it
  what     -> 2 kg rice, 1 packet tea
  how much-> dhai sau = 250
  terms    -> udhaar (credit), "likh lena" = write it down
```

Everything the ledger needs is in that message, already typed, by the customer, for free. The problem is therefore not data entry at all — the data exists and simply never reaches the ledger.

That reframing is what makes the product tractable. The job is not to build a better form; it is to build a bridge roughly three taps long between a message and a ledger row. It also explains why the hardest engineering in this project sits in two unglamorous places: reading another app's screen reliably, and parsing Hinglish numbers correctly.

WHY THE CONFIRM STEP SURVIVES DESPITE THE FRICTION ARGUMENT

Having argued that friction kills adoption, we then deliberately added a step: nothing is ever saved without a tap.

The reasoning is that the two frictions are not comparable. Typing an entry is repeated work; confirming one is a glance. And the alternative — an app that silently writes numbers into a ledger about money owed — fails the trust test that already kills the incumbents. One wrong silent entry costs more trust than a hundred confirmations cost effort.

PART II · SECTION 4

Nine decisions and the reasoning behind each

Each record below states the situation, the alternatives, the choice, the business reasoning, and what the choice cost. The last field is the one usually missing from decision logs, and the one that makes them worth re-reading.

DECISION 1

Overlay on top of WhatsApp, not a WhatsApp Business bot

The situation. The natural design is a bot: the shopkeeper forwards or messages an order to a bot number, it extracts the transaction and he confirms in the chat. It needs no new app surface and users already understand it.

Options considered. (a) WhatsApp Business API bot; (b) an Android overlay that reads the chat already on screen; (c) manual copy-paste into a standalone app.

Decision. (b) — a floating bubble drawn over WhatsApp that reads the chat via Android's accessibility API.

Business reasoning. The bot requires a verified WhatsApp Business Account. Verification needs business documentation and takes weeks with no committed date. For a project with a fixed graded deadline that puts a third party's approval queue on the critical path — an unbounded schedule risk we could neither estimate nor mitigate. There is also a recurring cost: WhatsApp Business messaging is priced per conversation, so a bot-based design carries a per-shop monthly cost that scales with usage before the product has revenue. The overlay has neither property: no approval, no per-message cost, and it works with the WhatsApp the shopkeeper already has.

What it cost. A large one, and it is structural. Accessibility-API apps cannot be distributed through the Play Store (section 14), so this decision traded a schedule risk for a distribution constraint. It also makes capture dependent on WhatsApp's internal view hierarchy, which is not a public contract and can change with any update. We judged an unbounded schedule risk worse than a known, mitigable distribution problem — a capstone that misses its deadline scores zero regardless of its architecture.

DECISION 2

Assisted reminders, not automated ones

The situation. Collection is the real commercial pain (section 1), so reminders are core. The obvious implementation sends them automatically.

Options considered. (a) automated sending via the WhatsApp Business API; (b) SMS via a gateway; (c) the app drafts the message and opens WhatsApp pre-filled for the shopkeeper to send.

Decision. (c) — compose the message with the amount and a UPI payment link, open WhatsApp pre-filled, and let him press send.

Business reasoning. Partly forced by decision 1: automated sending needs the same API we removed. But it is the better design independently. These messages are debt collection directed at the shopkeeper's own neighbours and regulars — the relationship is the asset, and a mistimed or wrongly-addressed automated message damages it permanently. Assisted sending means no message about money owed ever leaves the phone without a human seeing it. It also works with any customer today, with no opt-in, no template approval and no per-message fee.

What it cost. It does not scale to hundreds of reminders, and it cannot run unattended. For a shop with a few dozen credit customers that ceiling is far above real usage, so the constraint is theoretical at this stage.

DECISION 3

Deterministic rules first, the language model second

The situation. Extraction could be a single call to a capable model, which would handle Hinglish, quantities and payment terms with very little code.

Options considered. (a) model-only; (b) rules-only; (c) rules first, with the model filling only what rules cannot resolve.

Decision. (c).

Business reasoning. Three reasons that compound, and only one is about accuracy. Unit economics: extraction runs on every order message a shop receives — call it 30–60 a day. A model call on each is a recurring per-shop cost on a product whose customers are extremely price-sensitive; a rule costs nothing per call, and rules alone resolve the large majority of real messages. Availability: rules run with no network, and a shop counter frequently has no signal — a model-only design fails precisely when the shopkeeper needs it. Determinism: dhai sau is 250 every single time, whereas a model is 250 almost every time. This layer decides how much money goes into someone's ledger, and 'almost' is the wrong property for that.

What it cost. Considerably more code, and a lexicon of Hindi number words, payment keywords and item names to maintain. It also caps recall: rules cannot generalise to phrasings nobody anticipated, which is exactly the gap the model layer exists to close.

DECISION 4

The phone is the source of truth; sync is an optimisation

The situation. A server-authoritative design is simpler: one database, no conflict handling, no local schema to migrate.

Options considered. (a) server-authoritative with a thin client; (b) offline-first with the phone authoritative and background sync.

Decision. (b).

Business reasoning. Connectivity at a shop counter is unreliable, and the moments that matter most — mid-transaction, customer waiting — are exactly when a network round trip is least tolerable. A server-authoritative app is unusable at the moment of use, which is a product-defining failure rather than an inconvenience. Making the phone authoritative also means a backend outage degrades the product rather than stopping it.

What it cost. Sync must be idempotent, which forced client-generated transaction IDs and careful reasoning about replay. It also means the on-device schema needs real migrations, because that database holds the shopkeeper's money and cannot be dropped and rebuilt.

DECISION 5

Confidence is scored per field, not per entry

The situation. The engine needs to decide when to ask the user something. The simple approach is one confidence score per extracted entry.

Options considered. (a) a single blended score with one threshold; (b) a score per field, routing per field.

Decision. (b) — above 0.85 auto-fill, 0.50–0.85 ask about that specific field, below 0.50 fall back to buttons.

Business reasoning. 'The amount is certain but the item is a guess' is the common case, not the exception. Blended into one number that entry lands mid-range and the app asks about everything — which is precisely the form-filling that kills the incumbents (section 2). Per field, the app fills in the amount and asks one question about the item. That difference is the difference between a product used daily and one abandoned in a week.

What it cost. More complex scoring and a UI that has to express uncertainty without looking broken. The confirm sheet solves this by tinting only the weak field and giving its reason in plain language.

DECISION 6

Deduplication is a content hash, never a model

The situation. Bulk import can re-read messages already processed. Something must decide what is new. Since a model is already in the pipeline, asking it 'have I seen this before?' is tempting.

Options considered. (a) ask the model; (b) a per-chat cursor holding a timestamp and a rolling window of SHA-1 content hashes.

Decision. (b).

Business reasoning. A model that is 99% accurate at that question still double-counts one message in a hundred. At 30 messages a day that is a corrupted khata every few days, and — critically — the shopkeeper has no way to identify which entry is the wrong one. The error is silent and compounding. A hash comparison is exact, free, works offline, and survives re-importing an overlapping export. The model's job is the genuinely linguistic question: which of the new messages is an order?

What it cost. Effectively none. This is the clearest case in the project of choosing the boring mechanism because the failure mode of the clever one is unacceptable.

DECISION 7

The voice model never sees a number

The situation. The dashboard reads the day's summary aloud in Hindi. The obvious implementation hands the figures to a model and asks for a natural sentence.

Options considered. (a) model generates the sentence from the figures; (b) the database computes, a per-language template formats, and the voice model only reads a finished sentence.

Decision. (b).

Business reasoning. A model that renders ₹4,240 as 'about four thousand' has misstated a shopkeeper's own takings — and because he is listening rather than reading, he has no way to catch it. The template approach removes the opportunity entirely rather than trying to prompt around it. Amounts are spelled into words by the same number table the parser uses, run backwards, so the two directions cannot drift apart.

What it cost. Each supported language needs a hand-written template, which is why we ship three properly rather than eleven approximately. The Marathi template is explicitly marked as a draft pending review by a Marathi speaker (section 16).

DECISION 8

A zero-build dashboard, against our own plan

The situation. The project plan specified Next.js and a charting library for the web dashboard.

Options considered. (a) Next.js as planned; (b) a single HTML file with a hand-drawn SVG chart and no build step.

Decision. (b).

Business reasoning. The team is new to this stack and the dashboard is demo infrastructure, not the product. A broken `npm run build` twenty minutes before a viva costs more than a plainer chart does. Nothing server-side changes if it is ported later, so the decision is cheap to reverse.

What it cost. A deliberate departure from an approved plan, recorded here rather than quietly made. Charting is hand-rolled, so richer visualisations would mean more work than a library would need.

DECISION 9

Plain Android Views for the overlay, not Compose

The situation. The rest of the Android ecosystem has moved to Jetpack Compose, and using it would be the conventional choice.

Options considered. (a) Compose throughout; (b) plain Views for overlay windows.

Decision. (b) for everything drawn in a WindowManager overlay.

Business reasoning. Compose inside an overlay window requires ViewTreeLifecycleOwner and SavedStateRegistryOwner to be wired by hand, because there is no Activity providing them. That is real complexity and a real source of subtle bugs, and it buys nothing for a 54dp circle with a letter in it. The highest-risk component in the project is the overlay itself; keeping it boring was worth more than keeping it modern.

What it cost. More verbose UI code, and the eventual in-app screens will likely use Compose, leaving two UI idioms in one codebase.

PART III · SECTION 5

System overview

Three tiers. The phone is authoritative; the backend holds the engine and the shared view; the dashboard is a reader.

```
ANDROID  (Kotlin, 3,755 lines)
  OverlayService ....... the floating bubble, SYSTEM_ALERT_WINDOW
    |- BubbleView ...... draggable, snaps to the screen edge
    |- PanelView ....... today's figures, read from Room
    |- ConfirmSheetView  per-field confidence, "Sahi hai" / "Badlo"
    |- EditSheetView ... the human overrules the engine
    |- KhataView ....... balances, entries, reminder buttons
  capture/
    |- LedgerAccessibilityService   reads the chat window
    |- ChatScraper ..... node tree -> messages
    |- ImportExportActivity ....... WhatsApp "Export chat" target
  data/  Room: shops, customers, transactions, cursors, corrections
  sync/  SyncWorker: idempotent, client-generated UUIDs
        |
        |  REST over HTTP
        v
BACKEND  (Python, FastAPI + SQLite, 3,728 lines)
  /extract .......... rules -> Claude -> per-field confidence
  /ingest/export .... bulk chat import with watermark dedup
  /sync ............. idempotent, bidirectional
  /corrections ...... the learning loop
  /summary/narrate .. templated text -> Sarvam Bulbul v3
        |
        v
DASHBOARD  (one HTML file, no build step, served at /)
```

Every input path has an on-device fallback. Speech recognition runs locally, the rules layer runs locally, and the ledger is local. The consequence is that the core loop never requires a network — a network improves extraction quality and shares data, but its absence degrades the product rather than stopping it.

PART III · SECTION 6

The extraction engine

Turning “2 kilo chawal aur 1 packet chai bhej do, dhai sau ka udhaar likh lena” into a ledger row. Four stages, in order.

Stage	What it does	Why it is where it is
1 · Rules	Hindi number words, payment keywords, quantities, item names, sale vs payment direction. Regex and lookup tables.	Free per call, works offline, exactly repeatable. Resolves most real messages without a model.
2 · Model	Claude, given only the fields the rules could not settle, plus the shop's own customer and item vocabulary.	Handles phrasings nobody anticipated. Scoped narrowly so it cannot overrule arithmetic that already succeeded.
3 · Confidence	Per-field score from layer agreement, field completeness and name-match strength.	Decides what the shopkeeper is asked. Not a log line — it drives the interface.
4 · Names	Fuzzy match plus transliteration against the shop's customers.	So Rahul and राहुल are one customer, and the khata does not sprout duplicates.

The learning loop

Every time the shopkeeper taps Badlo and corrects a field, the pair (`raw_text` → `corrected_json`) is stored. Similar pairs are retrieved later and supplied to the model as few-shot examples for that shop specifically.

Per-shop matters commercially. One shop's chai is a ten-rupee packet; another's is a ninety-rupee box. A single shared model cannot be right for both, and a system that learns each shop's own vocabulary gets better the longer it is used — which is the retention mechanic as much as the accuracy mechanic.

WHY THE CORRECTION PATH HAD TO BE BUILT, NOT DEFERRED

For most of the project the Badlo button was a placeholder that dismissed the sheet. That single dead button quietly invalidated the entire learning claim: no corrections meant no training pairs, which meant nothing to learn from.

It is a useful reminder that a feature described in a plan and a feature on a live code path are different things, and only one of them is worth claiming.

PART III · SECTION 7

Hindi numbers, in both directions

Shopkeepers do not say “two hundred and fifty”. They say dhai sau. Hindi has a system of fractional multipliers with no English equivalent, and it is used constantly for money.

Spoken	Structure	Value
dhai sau	2.5 × 100	250
dedh sau	1.5 × 100	150
sava sau	(1 + ¼) × 100	125
paune do sau	(2 − ¼) × 100	175
sadhe teen sau	(3 + ½) × 100	350
do hazaar paanch sau	2×1000 + 5×100	2,500
ढाई सौ	Devanagari, same value	250

Hindi numerals are irregular all the way to a hundred, so there is no generating rule — 1 to 99 is a lookup table, with the many romanisations shopkeepers actually type (dhai, dhaai, adhai) mapped onto one value.

WHY THE PARSER ALSO RUNS BACKWARDS

The spoken dashboard needs ₹4,240 rendered as “chaar hazaar do sau chalees rupaye”. That is the same table read in the other direction.

Implementing both directions in one module means they cannot drift apart, and it enables a round-trip test: parse everything we can say, and say everything we can parse. That test catches divergence that would otherwise appear silently, months later, in a spoken figure nobody checked.

PART III · SECTION 8

Reading the chat

Path	Mechanism	Trade-off
A · Accessibility	Walks WhatsApp's view hierarchy and reads message text directly.	Exact text, no OCR errors, unaffected by FLAG_SECURE. Primary.
B · Screenshot + OCR	MediaProjection to a bitmap, then on-device ML Kit.	Roughly 85% on Devanagari, but works on any app — payment confirmations, bank SMS, a photo of a paper khata.

Why the parser leans on shape, not view IDs

The obvious way to find message bubbles is to match WhatsApp's view IDs. We deliberately do not depend on that. Those IDs are not a public contract — they change between versions, and a parser tuned to one build reads zero messages on the next, silently. A silent zero is the worst failure available here: the app looks healthy and simply stops recording income.

So the scraper reasons structurally:

- a message is a leaf node carrying real text
- an editable node is the compose box, never a message
- a clickable short label is a button, not a bubble
- outgoing bubbles sit right of centre, incoming left
- timestamps and date separators look like timestamps and dates
- messages are ordered by vertical position, because chat lists are commonly laid out reversed and traversal order is not chat order

TESTED AGAINST A REAL APP — AND WHAT THAT DOES NOT PROVE

This parser was originally written against an imagined hierarchy, which is the most dangerous kind of code: plausible, reviewable, and never executed.

WhatsApp cannot be installed on a bare emulator without a Play sign-in, so we tested against Google Messages driven by injected SMS — the same accessibility API and the same class of view tree. The parser went from reading 12 nodes (4 real messages plus 8 pieces of chrome) to exactly the 4 real messages, and those four fed to the engine produce ₹250 / udhaar / chawal 2kg + chai 1packet at 0.89 confidence.

This does not prove WhatsApp works. It proves the code path is sound and it shook out three real bugs. Running it against real WhatsApp on a physical phone remains the largest open unknown in the project, and is listed as such in section 16 rather than quietly assumed.

PART III · SECTION 9

Bulk ingestion and deduplication

A busy shop receives thirty orders across eight chats before lunch. Tapping capture thirty times is not a product, so whole conversations can be imported at once.

Lane	Mechanism	Why it exists
Export Chat	WhatsApp's own export, shared to us as a .txt	Official, complete, up to 40,000 messages, needs no permission at all
Scroll-harvest	Accessibility scrolls the chat backwards, reading as it goes	In place, triggered by voice — no app switch
Notification listener	Stages incoming messages as they arrive	The backlog is ready before he asks for it

```
10 messages
  -> watermark ....... drop anything already processed
  -> segment ......... group consecutive messages by sender (3 min window)
  -> classify ........ 8 blocks -> 4 candidates   (50% dropped as chatter)
  -> batch extract ... 4 blocks in ONE model call, not four
  -> review queue .... "Sab sahi hai" approves the confident ones at once
```

Segmentation matters more than it looks. Orders routinely arrive across three messages — “2 kilo chawal” / “aur 1 chai” / “udhaar likh lena” — and mean nothing apart. Grouping consecutive messages from one sender inside a three-minute window reassembles the order before extraction sees it.

The classifier that drops 50% is deliberately cheap and rule-based. Most chat traffic is “ok”, “thik hai”, “namaste”; filtering it before any model call is the difference between one API call and twenty, which is a direct operating cost (section 13).

PART III · SECTION 10

The ledger, sync and khata aging

Idempotent sync

Transaction IDs are generated on the phone, not assigned by the server. The phone is where the entry was confirmed, so the phone owns its identity. That single choice is what makes sync safe: replaying a batch after a dropped connection updates rows instead of creating a second set.

```
first send   ->  accepted=1  duplicates=0
identical replay ->  accepted=0  duplicates=1
row count unchanged
```

Khata aging, and a bug we nearly shipped

AGING MUST FOLLOW THE MONEY, NOT THE CALENDAR

“Days overdue” originally took the date of the oldest unpaid udhaar. On seeded data every single customer showed 85–89 days overdue, which no real shop looks like.

The cause: a customer who had cleared a debt months ago was still being aged from that cleared debt. Every regular customer therefore read as permanently overdue, the flag stopped carrying information, and the reminder it drives would have told someone “pending 85 days” about money they had already paid — the exact relationship damage decision 2 exists to prevent.

Payments now settle debts oldest-first, and a balance is aged from the oldest debt still uncovered. The seeded spread moved to 14–63 days. Worth noting the totals were correct throughout; only the rendered list revealed it.

This logic is deliberately duplicated in Kotlin rather than fetched from the server, because the khata must be correct offline. If the phone and the dashboard ever disagree about how many days someone is overdue, the shopkeeper stops trusting both — so the two implementations reference each other in comments and share the same test cases.

PART IV · SECTION 11

How we measure, and what the numbers mean

We built an evaluation harness: 122 messages, hand-labelled across ten categories, scored field by field. It runs in under a second and is re-run on every change.

Field	Precision	Recall	F1
amount	1.000	1.000	1.000
mode (cash / UPI / udhaar)	1.000	1.000	1.000
direction (sale / payment)	1.000	1.000	1.000
items	1.000	1.000	1.000
macro F1			1.000

Exact match across all four fields: 122 / 122 (100.0%).

READ THAT NUMBER CAREFULLY — AND SAY THIS BEFORE ANYONE ASKS

100% means the dataset is saturated, not that the engine is finished. We wrote these 122 cases, then fixed the engine until it passed them. The set no longer discriminates: it is a regression suite now, not a measure of capability.

The honest claim is “the engine handles every construction we have thought to test, and here is the list.” That is worth something — it is a great deal more than most projects can say — but it is not 100% accuracy in the wild, and this report does not say it is.

Two things follow, and both are planned: the next cases must be written by someone who did not write the engine, ideally drawn from real shopkeeper messages; and there must be a held-out split, because at present every case is a training case.

Why the baseline is the genuinely valuable artefact

Every figure above is rules-only, with the language model switched off. That is a real control group. When the model layer is measured, we will be able to state how much it helped rather than assert that it helps — the difference between an engineering claim and a marketing one. Very few projects at this scale retain a control condition, and it costs nothing to keep.

What measurement bought us

The harness is not decoration. Of the twenty-plus defects fixed in this project, nine were found by the evaluation harness alone — not by reading code, not by manual testing. The next section lists them by class, because the classes generalise and the individual bugs do not.

PART IV · SECTION 12

Every defect, and its lesson

Twenty-plus defects found and fixed, each with a regression test. They fall into five classes. For a system that records what people owe, the useful column is not what broke but what it would have cost.

Class 1 — words that are two things at once

BUG 01

“do” means 2, but usually means “send”

“chai bhej do, dhai sau” parsed as (2 + 2.5) × 100. Fixed with clause boundaries plus a verb-stem rule.

Real-world cost. A ₹450 entry instead of ₹250 — a 80% overcharge on a real customer, from the single most common word in an order message.

Lesson. A number word in a verb position is a verb. Hinglish is full of these and they must be resolved by context, not lexicon.

BUG 02

“diya” was in the item list

It is an oil lamp, and overwhelmingly the verb “gave”. So “phonepe kar diya” booked a lamp purchase.

Real-world cost. A phantom line item on every payment message — the ledger accumulating purchases that never happened.

Lesson. Recall is not worth an unbounded false-positive rate. We dropped the item entirely rather than defend it.

BUG 03

“daal do” is not lentils, “aata hoon” is not flour

daal and aata are real items, but “khaate mein daal do” means “put it in the account” and “kal aata hoon” means “I'll come tomorrow”.

Real-world cost. Groceries invented out of ordinary conversation, inflating what a customer appeared to owe.

Lesson. Ambiguous item words now require a quantity or unit beside them; unambiguous ones do not.

Class 2 — money that was not money

BUG 04

A quantity was reused as a price

“2 kilo chawal bhej do” booked ₹2. Every number had been consumed as a quantity, then the amount picker fell through and took one back.

Real-world cost. A ₹250 sale recorded as ₹2 — the shop losing 99% of the value of that transaction, silently.

Lesson. A missing amount is recoverable, because the app asks. A wrong one is money out of someone's pocket. The picker now returns nothing rather than guessing.

BUG 05

“X ka Y” means “Y worth X”

“paune sau ka chai” is seventy-five rupees of tea, not seventy-five teas. Read as a quantity, the price vanished into the item and the entry saved with no amount at all.

Real-world cost. Entries with an item and no value — an unrecorded sale, and an unrecoverable one once the message scrolls away.

Lesson. Grammar carries semantics. The Hindi possessive marks the number before it as money.

BUG 06

A measure was picked as the price

In “chai khatam ho gayi, 2 packet bhej do” a clause break stopped the 2 reaching chai, so it became a ₹2 sale.

Real-world cost. Same as bug 4: near-total loss of value on that transaction.

Lesson. A number directly followed by a unit is a measure, full stop — regardless of whether it found its item.

Class 3 — the tooling lied

BUG 07

\b does not work on Devanagari

Python excludes Devanagari vowel signs from `\w` because they are nonspacing marks, so a trailing `\b` never matches after a word ending in one — which is most Hindi words. Every Devanagari pattern in the rules layer was silently dead code.

Real-world cost. Devanagari messages — a stated product requirement — were handled far worse than the test suite implied, and nothing reported an error.

Lesson. Regex primitives assume Latin script. Verify them per script, with a test that would fail if the pattern matched nothing.

BUG 08

A timestamp used an invisible space

Android renders “11:07 PM” with U+202F, a narrow no-break space, which `\s` does not match. The timestamp filter silently missed.

Real-world cost. Timestamps captured as order text, corrupting extraction input.

Lesson. Same class as bug 7. When a string filter mysteriously fails, print the bytes.

BUG 09

Anthropic() constructs fine with no credentials

It only fails at request time. Treating construction as proof of availability made the evaluation print “rules + claude-opus-5” above numbers the model had never touched.

Real-world cost. A wrong conclusion heading straight into a report: “the model layer provided no measurable improvement”, when in fact it had never run.

Lesson. The eval now refuses to report stage-2 numbers when zero calls succeeded, and /health states which stages are live.

BUG 10

sort is not a version sort

`sort | tail -1` picked `gradle-8.2` over `gradle-8.13`, because `'2' > '1'`.

Real-world cost. The Android plugin rejected the old Gradle and the failure read like a code error, costing debugging time on healthy code.

Lesson. Use `sort -V`.

BUG 11

A filtered build log hid a failed build

Piping Gradle through `grep '^e:'` showed nothing, and the install reported “Success” — having deployed a twenty-minute-old APK.

Real-world cost. Twenty minutes spent debugging code that was not running on the device.

Lesson. Never filter a build log you have not first watched fail.

Class 4 — correct in code, wrong on screen

BUG 12

Every customer looked 87 days overdue

Aging ignored payments that had already cleared the debt (section 10).

Real-world cost. Overdue flags carrying no information, and reminders that would have told people they owed money they had paid.

Lesson. Some bugs are invisible in an API response and obvious in a rendered list of people. Look at the screen.

BUG 13

A new phone's first sync 404'd

Transactions and customers saved, but no `shops` row existed, so the dashboard could not find the shop. SQLite does not enforce foreign keys by default, so it succeeded silently.

Real-world cost. Indistinguishable from data loss on a user's first day — the single worst moment to look broken.

Lesson. Test the empty state, not only the seeded one.

BUG 14

The bubble drew, the letter inside it did not

`onMeasure` called `setMeasuredDimension` without measuring children.

Real-world cost. Cosmetic, but on the single most visible element in the product.

Lesson. Invisible in code review, obvious in a screenshot.

BUG 15

Capture read the app's own setup screen

`packageNames` in the accessibility config limits which apps send us events; it does not stop `rootInActiveWindow` returning a focused window from anywhere.

Real-world cost. A privacy failure as much as a correctness one — the product's central claim is that it reads WhatsApp and nothing else.

Lesson. Capture now enumerates windows for a supported chat window, wherever focus happens to be.

Class 5 — silent failures after apparent success

BUG 16

Sync failed after the server accepted the write

`SyncResponse.transactions` was typed as strings, but the server sends real transaction objects where amount is a number. The POST succeeded, the row saved, and parsing the response threw.

Real-world cost. The entry was on the server while the phone showed it pending forever — the worst shape a sync bug can take, because both sides look wrong in different ways.

Lesson. Type a response to what the server actually sends. The same mistake in /corrections was caught by this precedent.

BUG 17

Every sync job reported FAILED and posted nothing

`SyncWorker`'s constructor took an extra argument, and WorkManager's default factory constructs workers with exactly (Context, WorkerParameters).

Real-world cost. Entries never left the phone. The app looked entirely healthy; the failure lived only in a log nobody was reading.

Lesson. Take the dependency from a singleton rather than the constructor, unless a custom WorkerFactory is justified.

PART V · SECTION 13

Unit economics and cost model

The target customer is extremely price-sensitive. A design whose marginal cost scales with usage is a design that cannot be given away, and giving it away is how this category gets adopted.

Cost driver	Design response	Effect
Model calls per message	Rules resolve most messages with no model call; a rule-based classifier drops ~50% of chat traffic before extraction; bulk import batches N blocks into one request.	Marginal cost per order falls well below one model call
WhatsApp messaging fees	Assisted reminders via a wa.me deep link (decision 2).	Zero — no Business API, no per-conversation fee
Speech recognition	Android's on-device recogniser first; a hosted model only as an online upgrade.	Zero for the default path
Voice synthesis	Content-addressed audio cache keyed on text + language + speaker.	One synthesis per distinct summary, not per dashboard open
Backend hosting	SQLite and a single FastAPI process; Postgres only when multi-tenancy demands it.	Negligible at pilot scale

THE HONEST GAP IN THIS SECTION

These are design decisions that bound cost, not measured costs. The model layer and the voice model are both wired and credential-gated, so the actual rupees-per-shop-per-month figure is unmeasured.

Measuring it is a half-day of work once API keys exist, and it belongs in the next revision of this report rather than being estimated here. An invented number would be worse than an acknowledged gap.

PART V · SECTION 14

Distribution, and why the Play Store is closed

This is the direct cost of decision 1, and it is stated plainly rather than discovered by a reader.

Google restricts the accessibility APIs to applications whose core function is assisting users with disabilities. VaaniLedger uses them to read a chat screen for a shopkeeper's convenience, which is not that. An app using them for our purpose should expect rejection or removal from the Play Store.

Channel	Viability	Notes
Google Play	Closed	Accessibility policy. Not a technical obstacle — a policy one, so no amount of engineering changes it.
Direct APK / sideload	Viable now	Fits the actual distribution reality: these products are sold shop-to-shop by field agents who install on the spot.
Enterprise / MDM	Viable	Appropriate for a distributor or FMCG partner deploying across a network of shops.
Rebuild on Export Chat only	Viable, degraded	The chat-export lane needs no accessibility permission at all and would be Play-compliant, at the cost of the one-tap capture that makes the product distinctive.

The last row is worth noting as a genuine strategic option rather than a footnote: the bulk-import path alone is a viable, Play-distributable product with a slower interaction model. Keeping three independent capture lanes means that pivot stays available without an architectural rewrite.

PART VI · SECTION 15

What is true today

Three categories that a status table usually blurs, kept apart deliberately. Conflating them is how a project ends up claiming that compiled-but-never-called code works.

Verified — executed, with evidence

Component	Evidence
Extraction engine	189 automated tests; macro F1 1.000 on 122 labelled cases
Bulk import and dedup	10 messages → 8 blocks → 4 orders; re-importing the same export yields 0 new
Ledger and sync	Replay reports accepted=0, duplicates=1; row count unchanged
Web dashboard	Browser-verified in light and dark, at 375px, with a clean console
Spoken summary	386 characters of Hindi, amounts spelled out, within Bulbul's 2,500-character limit
The floating overlay	Draws over the home screen and other apps; survives leaving the app
Chat capture	Reads exactly the real messages off a live chat app's view tree, zero chrome
Full loop on device	Wiped database → panel shows ₹0 → capture → confirm → panel shows ₹250 sale / ₹250 udhaar, matching the server
Badlo → learning loop	Edit sheet prefilled, correction POSTed, pair stored: customer '' → 'Rahul Verma'
Khata screen	Kul baaki ₹250 · Rahul Verma ₹250 · entries with source tags
Assisted reminder	Draft composed and intent fired at com.whatsapp, with share-sheet fallback

Written but never executed

THIS MUST NOT BE DESCRIBED AS WORKING

ScreenCaptureService (screenshot + OCR) is a stub. MessageNotificationListener logs but does not yet stage to Room.

Voice: the recogniser launches correctly with hi-IN, but the transcript path has never run end to end, because this emulator has no speech backend. It needs a physical phone.

Unmeasured

The Claude extraction layer and Sarvam voice synthesis are both wired and credential-gated. Everything reported above is therefore the rules-only baseline. `/health` reports which stages are live, so a degraded run cannot be mistaken for a full one.

PART VI · SECTION 16

Risks and limitations we are not hiding

Risk	Severity	Mitigation / status
Capture untested against real WhatsApp	High	The largest open unknown. Proven against another chat app's view tree; WhatsApp's is its own. A one-minute diagnostic dump exists for when it breaks.
WhatsApp version drift breaks capture silently	High	Structural heuristics rather than view IDs reduce it. A “read zero messages” self-check that surfaces to the user is still needed.
OEM battery managers kill the overlay service	High	Xiaomi, Oppo and Vivo are aggressive. Untested on physical hardware — a real phone left running overnight is required.
No authentication	High	Anyone who knows a shop_id can read and write that shop's ledger. Disqualifying before any real pilot.
100% accuracy is on a saturated set	Medium	Stated in section 11. Needs externally-written cases and a held-out split.
Marathi template is a draft	Medium	Sentence frames are Marathi but amounts still render in Hindi number words. Must not ship without a Marathi speaker's review.
Play Store closed	Medium	Section 14. Sideload and enterprise are viable; an export-only build is a fallback.
Devanagari OCR at ~85%	Low	Accessibility is the primary path precisely because of this.
Overpayment not modelled	Low	Paying more than owed floors the balance at zero rather than carrying an advance.

WHY THIS SECTION EXISTS

Every one of these could have been omitted. A reader who finds a limitation we did not mention discounts everything else in the report; a reader who sees them named first has reason to trust the parts we do claim.

PART VI · SECTION 17

Roadmap

Immediate — before any demonstration

- Remove Google Messages from the accessibility scope; it is there only so the parser could be proven against a real view tree
- Disable the debug tree dump, which logs chat contents
- Move the API base address off the emulator-only 10.0.2.2

Next — completing the product

- Run capture against real WhatsApp on a physical phone
- Leave the overlay running overnight on an OEM device
- Measure the model layer against the rules-only control group
- Screenshot + OCR capture, and notification staging into Room
- Voice transcript path end to end on real hardware

Before a pilot with one real shopkeeper

- Authentication and an HTTPS deployment
- Tested Room migrations — that database holds his money
- Backup and export, so a lost phone is not a lost khata
- WhatsApp version-drift detection that surfaces to the user
- Crash reporting, and real error states throughout
- Evaluation cases written by someone outside the team

Before general availability

- A distribution decision from section 14
- Privacy policy and consent flow under India's DPDP Act
- Multi-tenant backend with monitoring
- A measured cost model (section 13)
- Onboarding that works with no developer present

PART VI · SECTION 18

How to run it

Backend, dashboard, tests and evaluation

```
# tests, accuracy evaluation and an end-to-end demo — no install needed
./run.sh

# the above, then the API and dashboard on :8000
./run.sh serve
```

Android

```
# builds, boots the emulator, installs and launches
cd android && ./build-and-run.sh

# then, in the app:
#   1. both permission cards should read CHALU HAI
#   2. tap "Bubble chalu karein"
#   3. press Home and open a chat — the bubble floats on top
#   4. tap the bubble, then "Yeh likh lo"
```

Measuring the model layer

```
export ANTHROPIC_API_KEY=sk-...
cd backend && .venv/bin/python -m eval.run --llm
```

Repository layout, module-by-module notes and the current status table are maintained in `README.md` and `docs/STATUS.md`, which are updated with the code rather than at the end.